



Universidade Federal de Santa Catarina – UFSC
DEC – Engenharia de Computação

Organização e Arquitetura de Computadores

Prof. Rodrigo Pereira, Dr.
adaptado de
Prof. Ney Calazans, Dr.

Última alteração: 11/08/2026

OS SEIS ELEMENTOS “DEFINITÓRIOS” DE UMA ARQUITETURA

1. *INTRODUÇÃO*
2. *OS REGISTRADORES ACESSÍVEIS AO PROGRAMADOR (EM ASSEMBLY LANGUAGE)*
3. *O CONJUNTO DE INSTRUÇÕES*
4. *OS FORMATOS DE INSTRUÇÃO*
5. *OS MODOS DE ENDEREÇAMENTO*
6. *A LINGUAGEM DE MONTAGEM (ASSEMBLY LANGUAGE)*
7. *O MODELO DE ACESSO À MEMÓRIA*

0 – INTRODUÇÃO

INSTRUCTION SET ARCHITECTURE - ISA

- *Uma arquitetura - é uma abstração de um (ou de uma classe de) processador(es)*
 - *Nome completo: Arquitetura do Conjunto de Instruções (em inglês Instruction Set Architecture ou ISA)*
- *Dominar uma ISA - saber programar o processador que ela descreve (em linguagem de montagem)*
- *Arquitetura **NÃO é Organização***
 - *E.g. – dezenas (centenas?) de organizações diferentes usam a arquitetura ARM v8!*

0 – INTRODUÇÃO

ORGANIZAÇÃO X ARQUITETURA

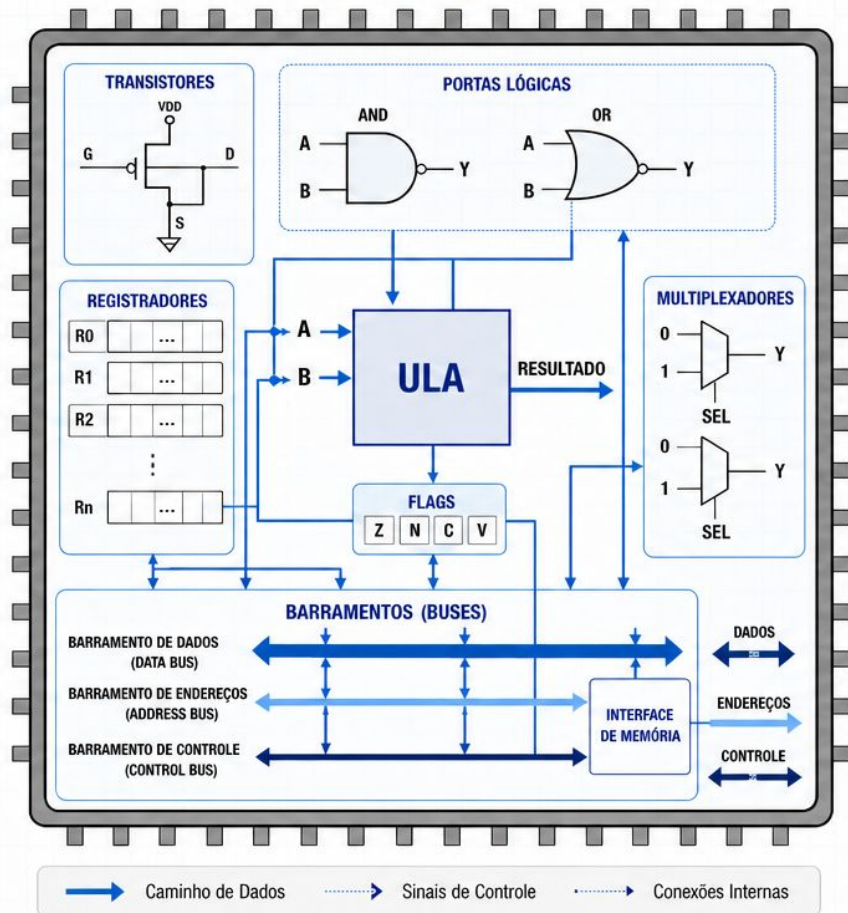
- **Organização de Processadores** – *A visão abstrata do engenheiro (eletricista, de computação) de um processador*
 - *Descrição contendo transistores e/ou portas lógicas e/ou registradores e/ou unidades lógico-aritméticas, fios, multiplexadores, barramentos etc.*
 - *Alguns Autores denominam Organização usando o termo Micro-Arquitetura, termo que o Autor acha ruim, mas muitos adotam esta nomenclatura...*
- **Arquitetura de Processadores** – *A visão abstrata do programador de baixo nível (linguagem de montagem, em inglês assembly language)*
 - *Descrição em termos de: instruções que o processador executa, registradores que o programador enxerga e usa para armazenar dados, a linguagem de programação de montagem etc.*

0 – INTRODUÇÃO

ORGANIZAÇÃO X ARQUITETURA

ORGANIZAÇÃO DE PROCESSADORES

Visão do Engenheiro



MESMO
HARDWARE,
DUAS
VISÕES

ARQUITETURA DE PROCESSADORES

Visão do Programador de Baixo Nível (Assembly)

REGISTRADORES VISÍVEIS AO PROGRAMADOR

R0	0x00000000
R1	0x00000005
R2	0x0000001A
R3	0x0000000F
R4	0x00001000
...	
Rn	0xFFFFFFFF

CONTADOR DE PROGRAMA (PC)

0x00000020

CONJUNTO DE INSTRUÇÕES
(OPCODES)

MOV	0x01
ADD	0x02
SUB	0x03
AND	0x04
ORR	0x05
LOAD	0x10
STORE	0x11
JMP	0x20
...	

CÓDIGO EM ASSEMBLY (EXEMPLO)

```
0x00000000 MOV R1, #5 ; R1 = 5
0x00000004 ADD R2, R1, R3 ; R2 = R1 + R3
0x00000008 LOAD R4, [0x1000] ; R4 = Mem[0x1000]
0x0000000C SUB R2, R2, #1 ; R2 = R2 - 1
0x00000010 JMP 0x00000020 ; Salta para 0x20
...
0x00000020 HALT ; Fim do programa
```

MODELO MENTAL DO PROGRAMADOR (CICLO DE INSTRUÇÃO)



0 – INTRODUÇÃO

DEFINIÇÃO DE PROCESSADOR

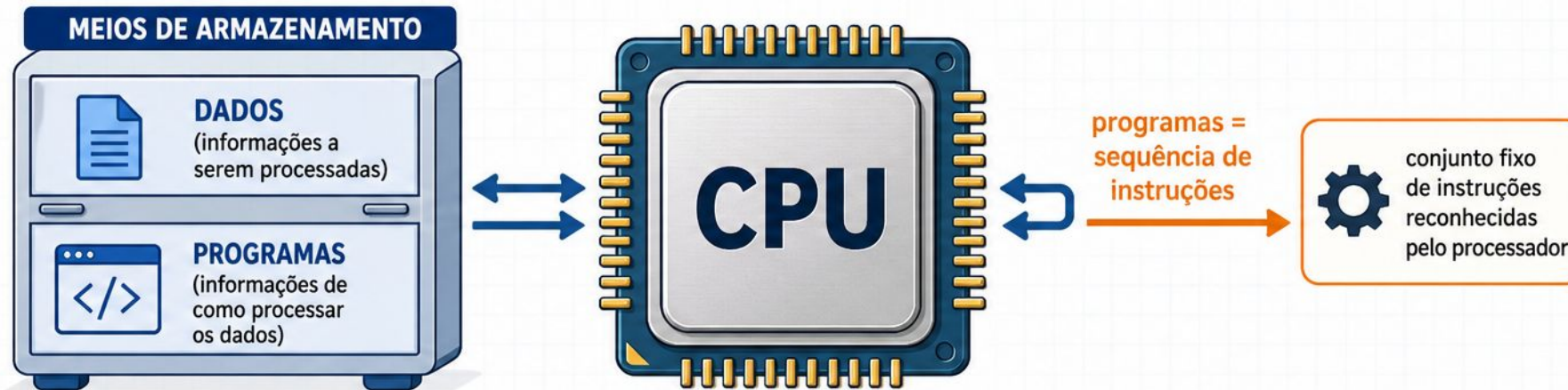
- *Mas afinal, o que é um **processador** (ou **computador**)? uma definição seria*
 - *Uma máquina com capacidade de acesso a meios de armazenamento onde estão estocadas informações a serem processadas e as informações que dizem como processar as primeiras. Também se chama o processador principal de um computador de **CPU** (do inglês Central Processing Unit)*
 - *As informações a serem processadas – são os dados*
 - *As informações de como processar dados – são os **programas***
 - ***Programas** – sequências de instruções, retiradas de um conjunto fixo de instruções reconhecidas como tal pelo processador*
- ***Funcionamento de um processador:** repetir, infinitamente, a sequência de 3 ações: (1) buscar instrução, (2) identificar a instrução buscada, (3) executar a instrução buscada*
- *A execução de instruções pode incorrer em acesso a dispositivos de entrada e saída*

0 – INTRODUÇÃO

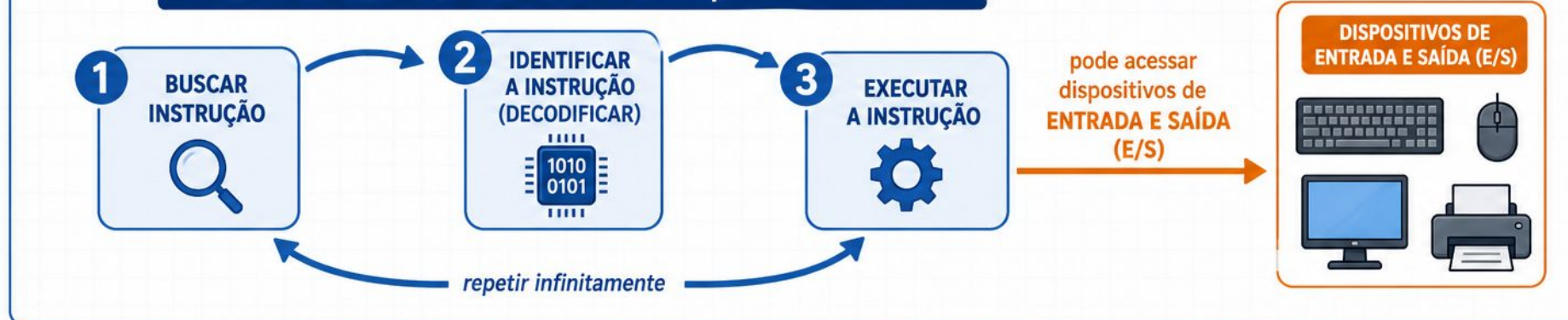
DEFINIÇÃO DE PROCESSADOR

O QUE É UM PROCESSADOR (CPU)?

Central Processing Unit — Unidade Central de Processamento



FUNCIONAMENTO DO PROCESSADOR – repetir infinitamente



0 – INTRODUÇÃO

DEFINIÇÃO DE PROCESSADOR

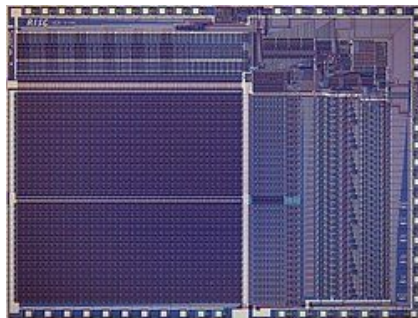
- *Alguns exemplos de ISAs usados hoje em dia - há dezenas, talvez centenas!!*
 - **x86** - arquitetura definida pela Intel e fabricada por ela, pela AMD etc.
 - **ARM v8** - arquitetura definida pela ARM e usada pela Samsung (no Galaxy S10), pela Apple (no iPhone 11) etc
 - **MIPS32 e MIPS64** - usadas pela Mobileye (Cia. da Intel para sensores automotivos), pela Tesla e pela Volvo em automóveis, em roteadores WiFi (da Cisco, etc.), aplicações em IA etc
 - **AVR** - Criada pela Atmel (da Microchip Tech desde 2016), usada em kits Arduino, em carros da BMW e Daimler-Chrysler, no controle do Xbox...
 - **MicroBlaze e Nios II**, propostas para uso em FPGAs (Xilinx e Altera, resp.)
 - **RISC-V** - Proposta na UC Berkeley - classe de ISAs abertas implementáveis (sem royalties) por qualquer pessoa/empresa. Google usa no Titan-M2 (segurança), em uso por múltiplas empresas (China, Índia etc)

0 – INTRODUÇÃO

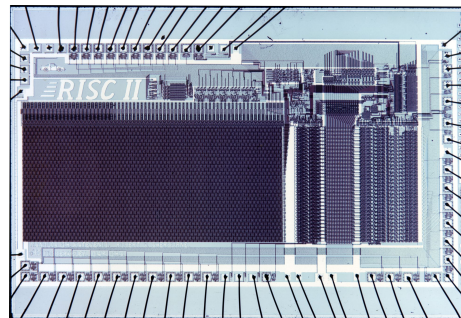
RISC-V Instruction Set Architecture



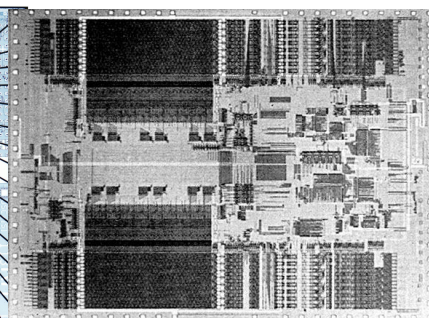
Berkeley Architecture Research



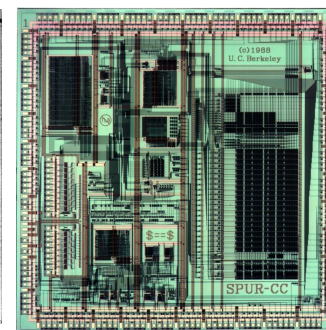
RISC I
(1982)



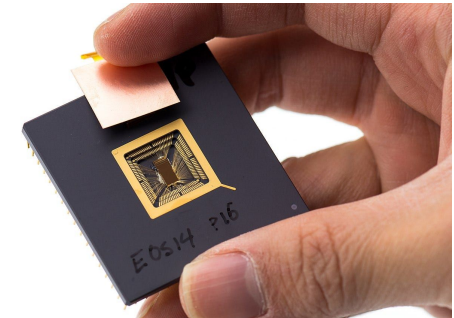
RISC II
(1983)



RISC III
SOAR
(1984)



RISC IV
SPUR
(1988)

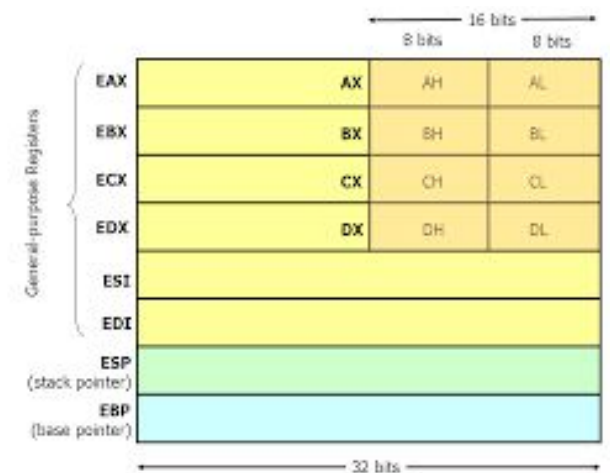
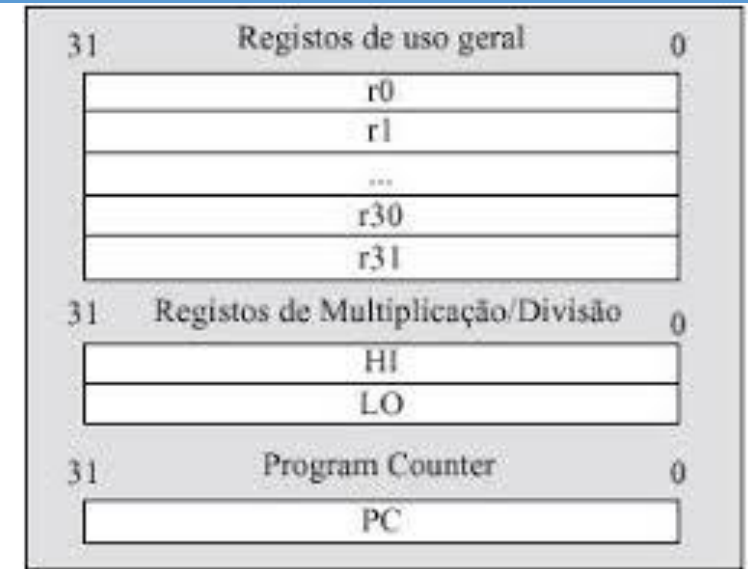


RISC V
(Maio de 2010 em diante)

FONTES: <https://people.eecs.berkeley.edu/~pattrsn/Arch/prototypes2.html>
<https://bar.eecs.berkeley.edu/projects/riscv.html>
<https://github.com/riscv/>

1 – OS REGISTRADORES ACESSÍVEIS AO PROGRAMADOR (EM LINGUAGEM DE MONTAGEM)

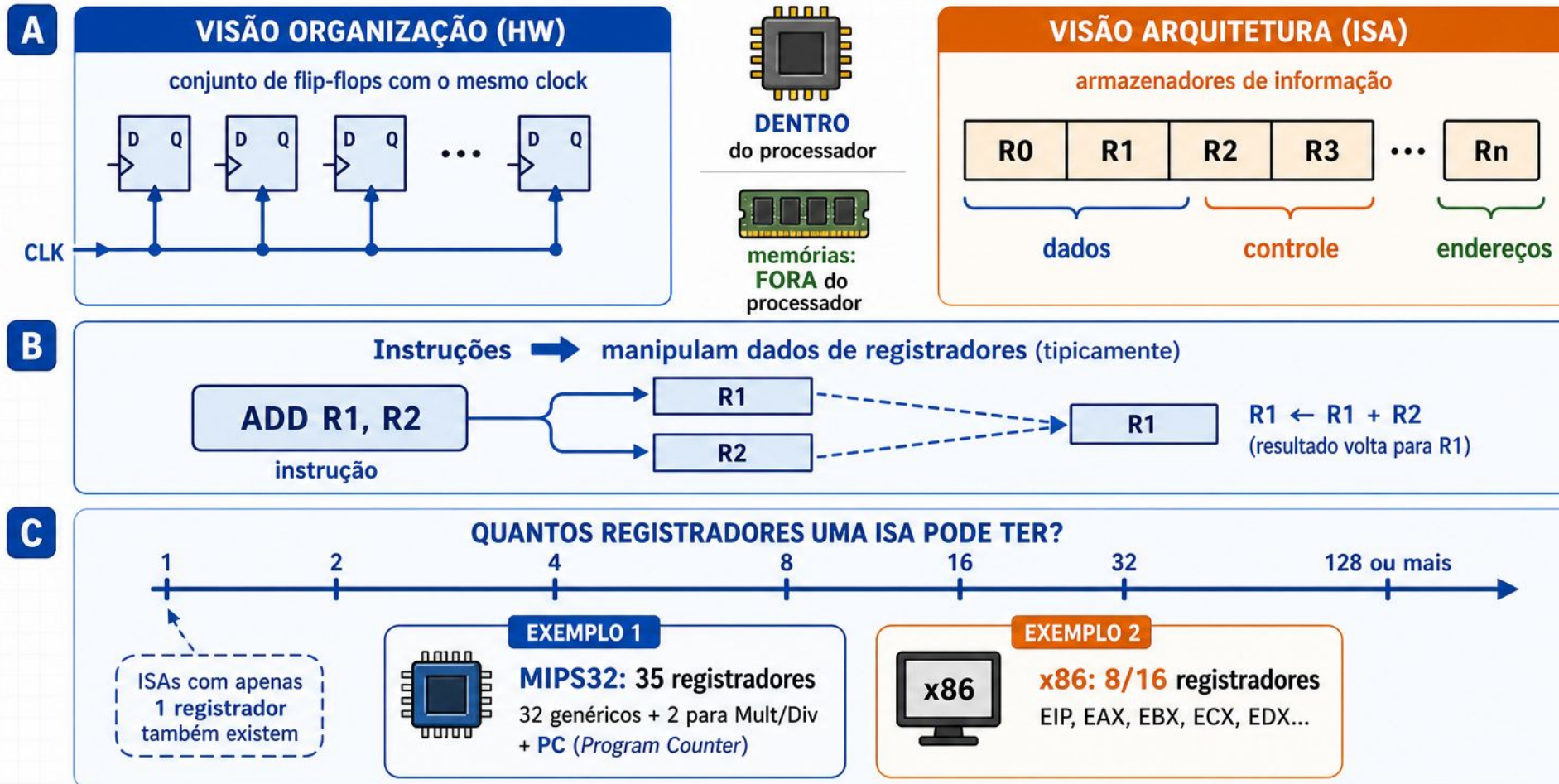
- **Registradores** - um dos elementos mais importantes do processador
- **Registradores** - visão de Organização (HW): conjunto de flip-flops com mesmo clock
- **Registradores** - visão Arquitetura (ISA): armazenadores de informação (dados, controle etc.)
- **Registradores** - dentro do processador (Memórias -> fora)
- **Instruções** - manipulam dados de registradores (tipicamente)
- Existem ISAs com apenas 1 registrador (pode haver mais em organizações específicas da ISA)
- Existem ISAs com 2, 4, 8, 16, 32, 128 ou até mais registradores
 - Exemplo 1: **MIPS32** possui 35 regs (32 genéricos + 2 para Mult/Div + PC)
 - Exemplo 2: x86 possui 8/16 registradores



1 – OS REGISTRADORES ACESSÍVEIS AO PROGRAMADOR (EM LINGUAGEM DE MONTAGEM)

REGISTRADORES – ELEMENTO CENTRAL DO PROCESSADOR

★ um dos elementos mais importantes do processador



2 – O CONJUNTO DE INSTRUÇÕES

- **Instruções** - o elemento “definitório” mais importante do processador
- **Visão arquitetural (ISA):** uma instrução - sequência de operações atômicas que alteram registradores e/ou memória
- **Cada ISA:** um número fixo de instruções (14, 36, 536, varia muito e há exceções...)
- **Instruções classificáveis segundo vários critérios (função, tamanho, duração, no. de operandos etc.)**
 - Exemplo: uma parte do conjunto de instruções do MIPS-I-(figura ao Lado)

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1,\$s2,\$s3	$s1 = s2 + s3$	Three operands; data in registers
	subtract	sub \$s1,\$s2,\$s3	$s1 = s2 - s3$	Three operands; data in registers
Data transfer	load word	lw \$s1,100(\$s2)	$s1 = \text{Memory}[s2 + 100]$	Data from memory to register
	store word	sw \$s1,100(\$s2)	$\text{Memory}[s2 + 100] = s1$	Data from register to memory
Logical	and	and \$s1,\$s2,\$s3	$s1 = s2 \& s3$	Three reg. operands; bit-by-bit AND
	or	or \$s1,\$s2,\$s3	$s1 = s2 s3$	Three reg. operands; bit-by-bit OR
	nor	nor \$s1,\$s2,\$s3	$s1 = \sim (s2 s3)$	Three reg. operands; bit-by-bit NOR
	and immediate	andi \$s1,\$s2,100	$s1 = s2 \& 100$	Bit-by-bit AND reg with constant
	or immediate	ori \$s1,\$s2,100	$s1 = s2 100$	Bit-by-bit OR reg with constant
	shift left logical	sll \$s1,\$s2,10	$s1 = s2 \ll 10$	Shift left by constant
	shift right logical	srl \$s1,\$s2,10	$s1 = s2 \gg 10$	Shift right by constant
Conditional branch	branch on equal	beq \$s1,\$s2,L	if ($s1 == s2$) go to L	Equal test and branch
	branch on not equal	bne \$s1,\$s2,L	if ($s1 != s2$) go to L	Not equal test and branch
	set on less than	slt \$s1,\$s2,\$s3	if ($s2 < s3$) $s1 = 1$; else $s1 = 0$	Compare less than; used with beq, bne
	set on less than immediate	slt \$s1,\$s2,100	if ($s2 < 100$) $s1 = 1$; else $s1 = 0$	Compare less than immediate; used with beq, bne
Unconditional jump	jump	j L	go to L	Jump to target address

3 – OS FORMATOS DE INSTRUÇÃO

Exemplo introdutório

Um exemplo de programa escrito em linguagem de montagem do MIPS

```
ADDIU    $29, $29, -32
SW       $31, 20($29)
SW       $4, 32($29)
SW       $5, 36($29)
SW       $0, 24($29)
SW       $0, 28($29)
LW       $14, 28($29)
LW       $24, 24($29)
MULTU    $14, $14
ADDIU    $8, $14, 1
SLTI     $1, $8, 101
SW       $8, 28($29)
MFLO     $15
ADDU     $25, $24, $15
BNE      $1, $0, -9
SW       $25, 24($29)
LUI      $4, 4096
LW       $5, 24($29)
JAL      1048812
ADDIU    $4, $4, 1072
LW       $31, 20($29)
ADDIU    $29, $29, 32
JR       $31
MOVE     $2, $0
```

O código-objeto do programa à esquerda, pronto para ser executado se carregado em memória

Pergunta - como o texto à esquerda se traduz nos bits à direita?
Resposta - através dos formatos de instrução!

```
0010011110111101111111111111100000
1010111110111111100000000000010100
1010111110100100000000000001000000
1010111110100101000000000001001000
10101111101000000000000000011000
10101111101000000000000000011100
10001111101011100000000000011100
10001111101110000000000000011000
00000001110011100000000000011001
00100101110010000000000000000001
00101001000000010000000001100101
10101111101010000000000000011100
00000000000000000111100000010010
00000011000011111100100000100001
00010100001000001111111111110111
10101111101110010000000000011000
00111100000001000001000000000000
10001111101001010000000000011000
00001100000100000000000001101100
00100100100001000000010000110000
10001111101111110000000000010100
00100111101111010000000000010000
00000011111000000000000000010000
000000000000000000000100000100001
```

3 – OS FORMATOS DE INSTRUÇÃO

Tomemos o código fonte da primeira instrução: **ADDIU \$29, \$29, -32**

O formato da instrução definida por ADDIU, retirado do manual de instruções do MIPS é:

Adição com dado imediato (sem overflow)

Formato Fonte:

Formato Objeto:

addiu rt, rs, imm

9	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

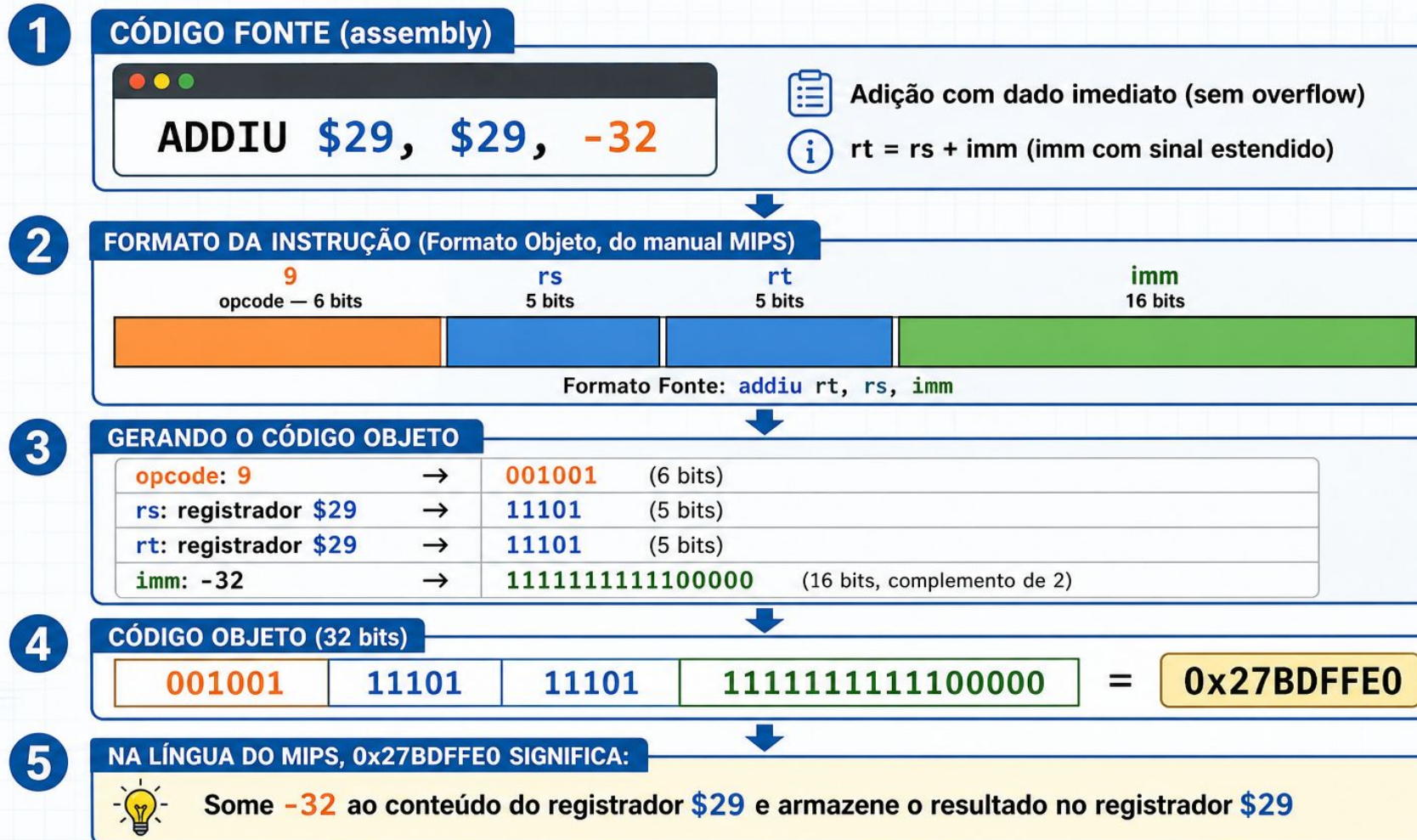
Semântica (o que isso quer dizer - semantikós):

Coloca a soma do registrador rs e do dado imediato (imm) com sinal estendido no registrador rt.

Com esta informação é possível gerar o código objeto a partir do código fonte, pois

- A constante 9 em binário representada em 6 bits é 001001
- O número do registrador rt (\$29) em 5 bits é 11101.rs é o mesmo registrador \$29
- -32, representado em complemento de 2 em 16 bits é: 1111111111100000
- Unindo-se estes 4 valores, obtém-se o código objeto da instrução, que é
 - 00100111101111011111111111100000, que representado de forma mais compacta, em hexadecimal é **0x27BDFFE0**
- Na língua do MIPS, **0x27BDFFE0** significa então: **“Some -32 ao conteúdo do registrador \$29 e armazene o resultado no registrador \$29 e não verifique o overflow”**

3 – OS FORMATOS DE INSTRUÇÃO



Em **ADDIU** - a letra "U" não significa que os operandos são tratados como números sem sinal (de *unsigned*), mas sim uma operação que não verifica overflow, como se os valores fossem tratados "sem preocupação com sinal".

3 – OS FORMATOS DE INSTRUÇÃO

Agora pode-se generalizar a discussão de formatos de instrução

- Formatos de instrução estabelecem uma relação unívoca entre o que o programador escreve (em linguagem de montagem) e o código executável pelo processador
- Cada instrução está associada um nome **mnemônico** e um **conjunto de operandos**
- Operandos podem ser referências a registradores, a constantes, referências a posições de memória, referências indiretas, etc. O modo de endereçamento associado a cada operando determina como obter o operando durante a execução da instrução
- Cada instrução pode ter um ou mais formatos associados a ela. Em máquinas RISC, frequentemente cada instrução possui um único formato
- A parte constante do formato (por exemplo o 9 no campo de 6 bits mais à esquerda da ADDIU, denominado de **opcode** ou **código da operação**, ver slide anterior) determina a instrução, enquanto que a parte variável (rs, rt, imm na ADDIU) determina um ou mais operandos desta

4 – OS MODOS DE ENDEREÇAMENTO

- Define-se modos de endereçamento como “**a forma como o processador localiza (recupera) cada operando de uma instrução**”
- Exemplo -> a instrução **BNE \$1, \$0, -9** do exemplo de programa MIPS apresentado antes possui 3 operandos: \$1, \$0 e -9. O caractere \$ identifica que os dois primeiros operandos são referências a **registradores** do **processador**. Se observarmos o formato desta instrução no manual, tem-se

Salta (**branch**) se for diferente

Formato Fonte:

bne rs, rt, imm

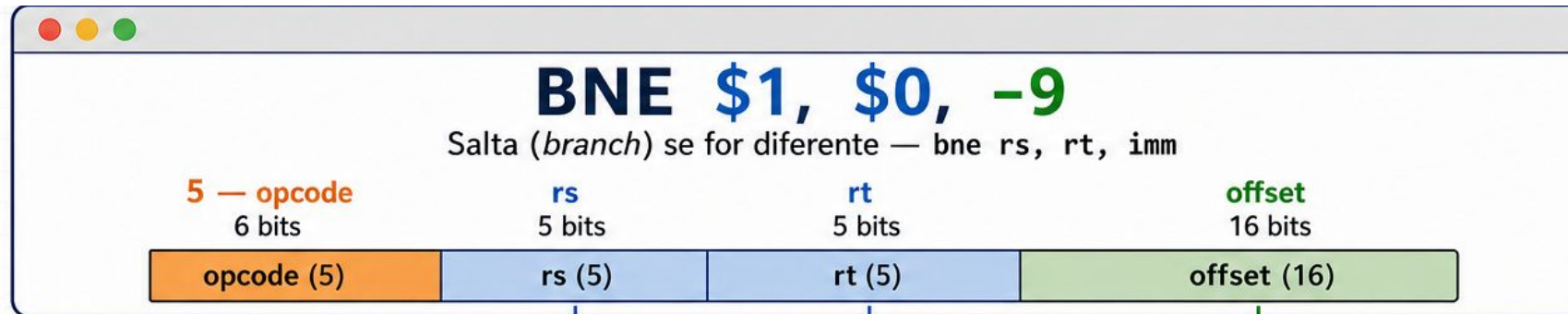
Formato Objeto:

5	rs	rt	offset
6 bits	5 bits	5 bits	16 bits

Semântica: Se o registrador *rs* contiver valor diferente do valor armazenado no registrador *rt*, pule o número de instruções designado pelo *offset* (**para a frente** se o *offset* for **positivo**, **para trás** se o *offset* for **negativo**), a partir da instrução que segue a instrução *bne*

- Os três operandos desta instrução **usam dois modos de endereçamento distintos**. Os dois primeiros (\$1 e \$0) usam o **modo a registrador**, pois os bits dos campos *rs* e *rt* representam números de registradores que contêm dados a usar na execução do *bne*. O último operando usa o **modo relativo (ao PC)** de endereçamento, o operando é o deslocamento (*offset*) em instruções para chegar à próxima instrução a ser executada, em relação à instrução seguinte ao *bne*, caso a comparação dos conteúdos dos registradores revela que estes são diferentes

4 – OS MODOS DE ENDEREÇAMENTO



SEMÂNTICA: Se rs contiver valor diferente de rt, pule o número de instruções designado pelo offset (para a frente se positivo, para trás se negativo), a partir da instrução que segue o bne



MODO A REGISTRADOR

(operandos \$1 e \$0)

ARQUIVO DE REGISTRADORES

\$0	0
\$1	dado em \$1
\$2	
...	...
\$31	

\$1 e \$0: os campos rs e rt contêm números de registradores que guardam os dados a comparar — o dado está dentro do processador.



MODO RELATIVO AO PC

(operando -9)

ENDEREÇO

N - 10
N - 8
N - 4
N (PC)
alvo: N + 4 (PC + 4)
N + 8
...

INSTRUÇÃO

...
...
...
BNE \$1, \$0, -9
instrução seguinte
...
...

$$\text{Alvo} = (\text{PC} + 4) + (\text{offset} \times 4)$$

o **offset** é um deslocamento em instruções, medido a partir da instrução seguinte ao bne (PC+4).

★ mesma instrução, dois modos de endereçamento

4 – OS MODOS DE ENDEREÇAMENTO

- Os modos de endereçamentos mais comumente encontrados na grande maioria das arquiteturas são os seguintes:
 - **Modo Imediato** -> o operando é uma constante contida dentro do código da instrução, estando assim imediatamente disponível para uso, o que justifica o nome do modo
 - **Modo a Registrador** -> o operando é uma referência a (i.e. um endereço de) um registrador interno do processador
 - **Modo Direto ou Absoluto** -> o operando é uma referência a (i.e. um endereço de) uma posição da memória externa ao processador (e.g. não existe no MIPS)
 - **Modo Relativo** -> o operando é um valor a ser adicionado a um registrador existente para obter o endereço de um operando (no exemplo do MIPS, o modo relativo é relativo ao valor do registrador program counter, PC, do MIPS)
 - **Modo Indireto** -> o operando é o endereço do endereço de um dado (muito útil para manipular vetores e matrizes em linguagem de montagem)
 - **Modo Base-Deslocamento** -> usa-se aqui dois valores para permitir localizar o operando: o primeiro é um endereço base (em um registrador ou em uma posição de memória), o segundo é uma constante a ser somada com o endereço base para obter o endereço do operando da instrução

4 – OS MODOS DE ENDEREÇAMENTO

1 MODO IMEDIATO

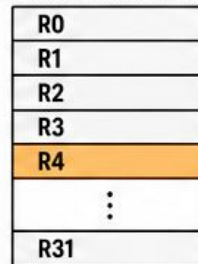
INSTRUÇÃO (PALAVRA)



o operando é uma constante contida dentro do código da instrução — imediatamente disponível para uso

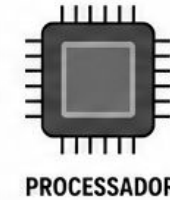
2 MODO A REGISTRADOR

BANCO DE REGISTRADORES



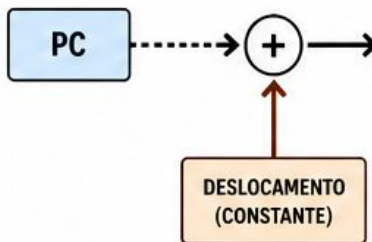
o operando é uma referência (endereço) a um registrador interno do processador

3 MODO DIRETO OU ABSOLUTO



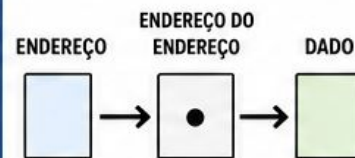
o operando é uma referência a uma posição da memória externa ao processador (ex.: não existe no MIPS)

4 MODO RELATIVO



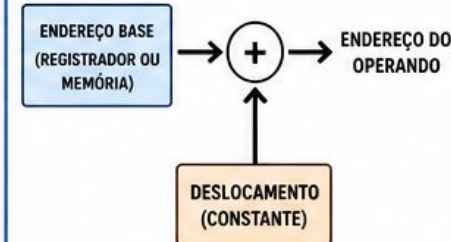
o operando é um valor somado a um registrador para obter o endereço — no MIPS, relativo ao PC

5 MODO INDIRETO



o operando é o endereço do endereço de um dado — útil para vetores e matrizes em linguagem de montagem

6 MODO BASE-DESLOCAMENTO



usa dois valores: um endereço base (em registrador ou memória) + uma constante somada para obter o endereço do operando

4 – OS MODOS DE ENDEREÇAMENTO

Exemplos de Assembly vs Modos de Endereçamento

- **Modo Imediato** - a constante está dentro da instrução
 - `addiu $t0, $t0, 10` # soma 10 ao conteúdo de \$t0
 - `ori $t1, $zero, 0xFF` # \$t1 = 0xFF (constante embutida na instrução)
- **Modo a Registrador** - operandos são referências a registradores
 - `add $t0, $t1, $t2` # \$t0 = \$t1 + \$t2
 - `sub $s0, $s1, $s2` # \$s0 = \$s1 - \$s2
- **Modo Direto ou Absoluto** - endereço de memória (não existe no MIPS)
 - A instrução seria algo como (em x86, para comparação): `MOV AX, [1234h]` ;
x86: busca o dado na posição de memória 1234h
 - MIPS oferece como aproximação é montar o endereço absoluto com duas instruções
 - `lui $t0, 0x1001` # carrega a parte alta do endereço
 - `ori $t0, $t0, 0x0000` # completa a parte baixa → \$t0 = 0x10010000

4 – OS MODOS DE ENDEREÇAMENTO

Exemplos de Assembly vs Modos de Endereçamento (cont.)

- **Modo Relativo ao PC** - o desvio conta a partir de PC+4
 - **bne** \$t0, \$t1, **ROTULO** # se \$t0 != \$t1, salta para **ROTULO**
... instruções ...
ROTULO:
addiu \$t0, \$t0, 1
- **Modo Indireto** - endereço do endereço
 - O MIPS não tem indireção dupla nativa; o padrão é feito com dois loads encadeados:
 - lw \$t1, (\$s0) # \$t1 = conteúdo de memória no endereço guardado em \$s0
(agora \$t1 contém o endereço do dado)
lw \$t2, (\$t1) # \$t2 = dado apontado pelo endereço apontado por \$s0

4 – OS MODOS DE ENDEREÇAMENTO

Exemplos de Assembly vs Modos de Endereçamento (cont.)

- **Modo Base-Deslocamento** - registrador base + constante
 - Modo mais usado na prática: com \$s0 apontando para o início de uma *struct* ou *vetor*, o deslocamento constante seleciona o campo ou o elemento. A soma base + deslocamento acontece dentro do processador, na unidade de cálculo de endereços
 - `lw $t0, 4($s0) # $t0 = memória no endereço (conteúdo de $s0) + 4`
 - `sb $t1, 12($s1) # armazena um byte em ($s1) + 12`

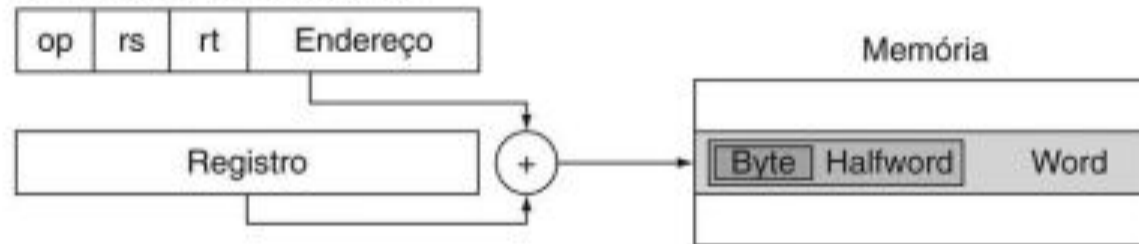
1. Endereçamento imediato



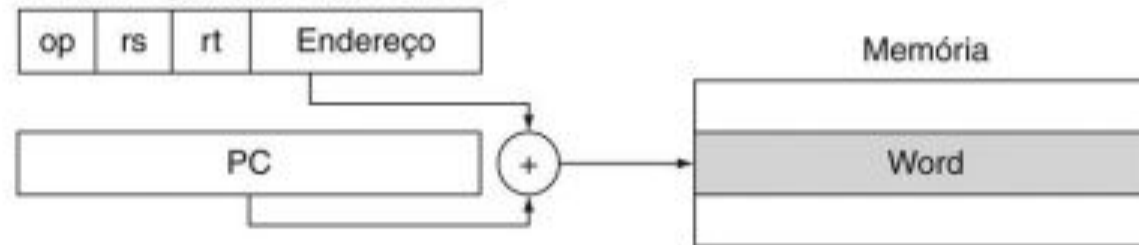
2. Endereçamento em registrador



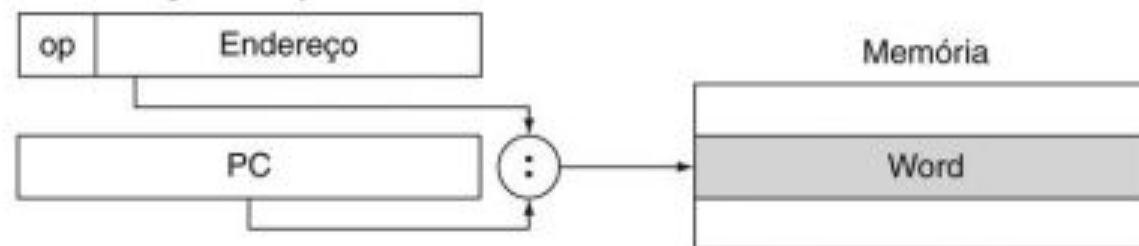
3. Endereçamento de base



4. Endereçamento relativo ao PC



5. Endereçamento pseudodireto



4 – OS MODOS DE ENDEREÇAMENTO

Exemplos de Assembly vs Modos de Endereçamento (cont.)

Observações

- Cuidado com documentação de fabricantes:
 - *De forma confusa algumas destas costumam associar modos de endereçamento a instruções e não a cada operando desta, como deveriam*
 - Por exemplo, uma instrução **ADDIU \$29, \$29, -32** seria normalmente classificada, em tais documentos, como uma instrução que usa endereçamento imediato, ignorando que os dois primeiros operandos usam o modo de endereçamento a registrador. Isto é péssimo e confuso, mas é a vida real
 - Portanto, são dois modos na mesma instrução
 - Está com dúvida? - sempre pergunte por operando: "onde está cada operando desta instrução?" — registrador, memória, imediato, PC?

5 – A LINGUAGEM DE MONTAGEM (ASSEMBLY LANGUAGE)

- **Assembly** - trata-se de uma **linguagem de programação**
- Existe uma **linguagem de montagem** diferente para cada arquitetura (ISA)
- Como em qualquer linguagem existem **comandos** -> as instruções da arquitetura
- Como em qualquer linguagem existem **dados** -> basicamente números de várias classes, textos (caracteres e/ou cadeias de caracteres), e menções a conteúdos de estruturas específicas, como registradores
- Além de comandos e dados, existem **diretivas de montagem** -> servem para definir dados, alinhamento de código, macros etc.
- Como em poucas linguagens, pode-se referenciar **registradores** (específicos) da **arquitetura** (e de qualquer organização desta) em programas
- **Programas são sequências de instruções**
- Instruções possuem formatos próprios (mnemônicos, sintaxe de operandos etc.)
- A seguir, dá-se um exemplo de um programa completo com comentários, diretivas de montagem, sequência de instruções e definições de dados

5 – A LINGUAGEM DE MONTAGEM

Exemplo C

```
#include <stdio.h>
```

```
static unsigned int array[11] = { 0x12, 0xff, 0x3, 0x14, 0x878, 0x31, 0x62, 0x10, 0x5, 0x16, 0x20 };  
static const unsigned int size = 11; static const unsigned int const_ = 0x100;
```

```
int main(void) {
```

```
    unsigned int *p; unsigned int n; unsigned int c; unsigned int tmp;  
    for (unsigned int i = 0; i < size; i++) {printf("array[%u] = 0x%X\n", i, array[i]);}
```

```
    p = array; n = size; c = const_;
```

```
    while (n > 0) {  
        tmp = *p;  
        tmp = tmp + c;  
        *p = tmp;  
        p++; n--;  
    }
```

```
    for (unsigned int i = 0; i < size; i++) { printf("array[%u] = 0x%X\n", i, array[i]);}  
    return 0;  
}
```

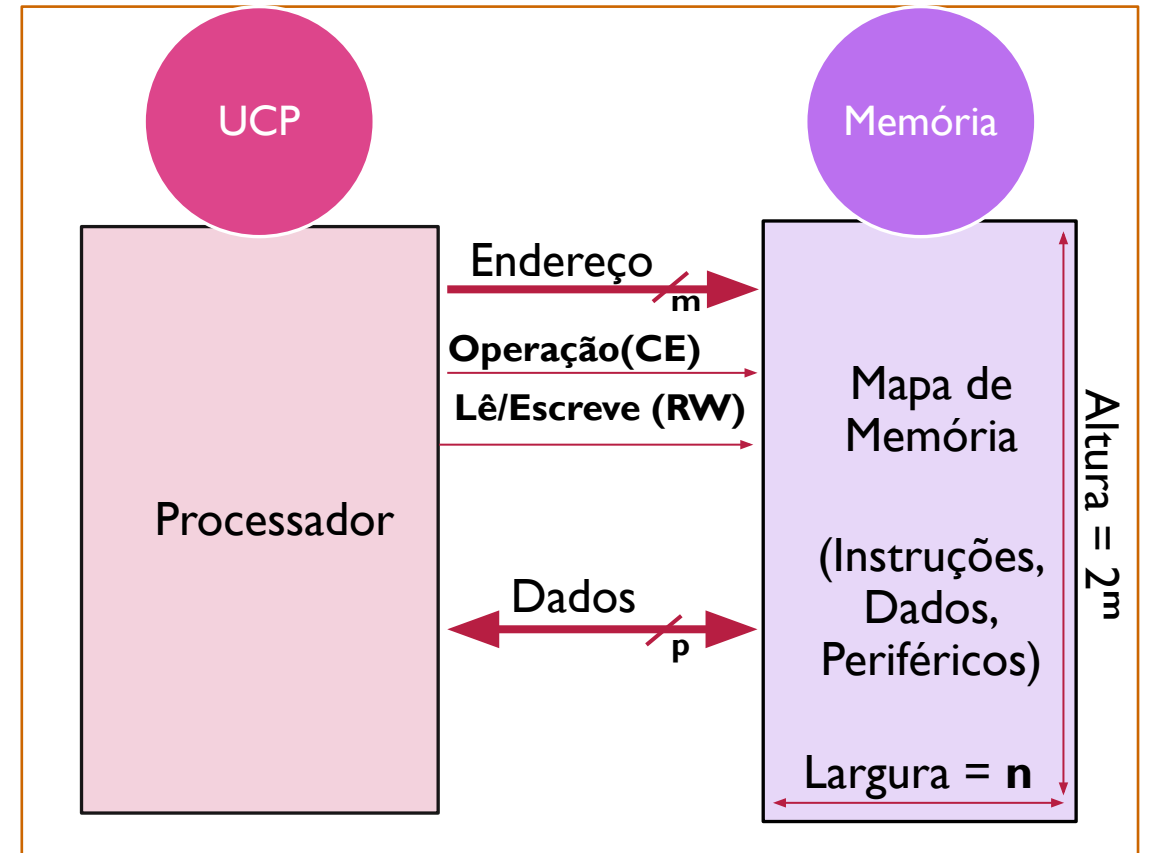
```
#####
# #####
# # Exemplo de programa completo em linguagem de montagem do MIPS : Manipulando vetores
# # Autor: Ney Calazans
# # Função: Este programa soma o valor de uma constante (const) a cada elemento do vetor array
#####

        .text                # Diretiva para o montador - adiciona o que vem abaixo
                                #      à memória de programa do processador
        .globl  main          # Diretiva - Declara o rótulo main como sendo global
m      #      main é o ponto a partir de onde se inicia a execução
main:
        la        $t0,array    # Pseudo: O registrador $t0 contém o endereço do vetor array
        la        $t1,size     # Obtém o endereço da posição da memória de dados onde se guarda
                                #      o tamanho do vetor
        lw        $t1,0($t1)   # O registrador $t1 carregado com o tamanho/nro elementos do vetor a tratar
        la        $t2,const    # Obtém o endereço da constante const
1      lw        $t2,0($t2)     # O registrador $t2 contém a constante a somar
loop:   blez      $t1,end      # Se o elementos a tratar é (ou chega a) 0, fim do processamento
        lw        $t3,0($t0)   # Senão, busca na memória o próximo elemento do vetor
        addu      $t3,$t3,$t2   # Soma o elemento atual com a constante
        sw        $t3,0($t0)   # Atualiza o vetor em memória
        addiu     $t0,$t0,4     # Faz o apontador do vetor ($t0) apontar para o próximo elemento
                                #      lembrar que 1 palavra no MIPS ocupa 4 endereços consecutivos de memória
        addiu     $t1,$t1,-1    # Decrementa o número de elementos que resta tratar do vetor
        j         loop         # Volta a executar novamente o teste de permanência no laço
e      #      Ao final do laço, volta-se para o Sistema Operacional (SO)
end:    li        $v0,10        # Prepara chamada do sistema para finalizar programa
        syscall               # Instrução de chamada do SO - Produz o fim do programa

        .data                # Diretiva para o montador - adiciona o que vem abaixo
a      #      à memória de dados do processador
array:  .word     0x12 0xff 0x3 0x14 0x878 0x31 0x62 0x10 0x5 0x16 0x20 # O vetor de dados em si
                                #      A diretiva .word reserva 4 bytes (uma palavra) para cada inteiro,
s      #      em posições sucessivas da memória de dados
c size:  .word     11          # Variável que armazena o tamanho do vetor (no caso, 11 elementos)
const:  .word     0x100        # Constante a somar a cada elemento do vetor
```

6 – O MODELO DE ACESSO À MEMÓRIA

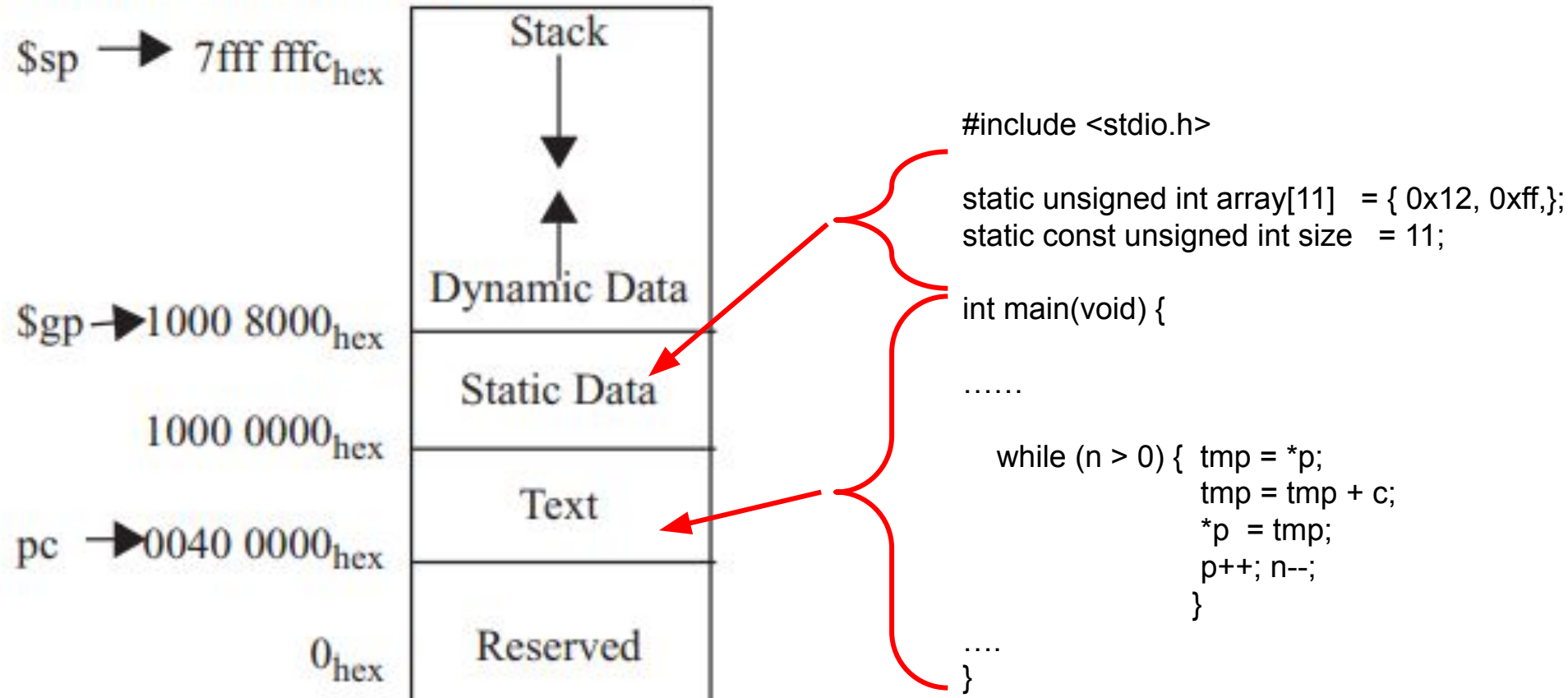
- Fora do processador existe apenas a Memória (visão da Arquitetura)
- Processador acessa a Memória lendo ou escrevendo nela
- Endereço – valor em m fios, define que a Altura da memória é 2^m
- Largura da memória n – unidade de acesso mínimo à memória
- Dados – Conjunto de p fios que trazem/levam dados de/para a memória. Note-se p pode ser igual a n ou diferente de n -> p costuma ser múltiplo de n
- Mapa de memória – Uma abstração $\square$ define a memória como uma tabela de $2^m \times n$ bits



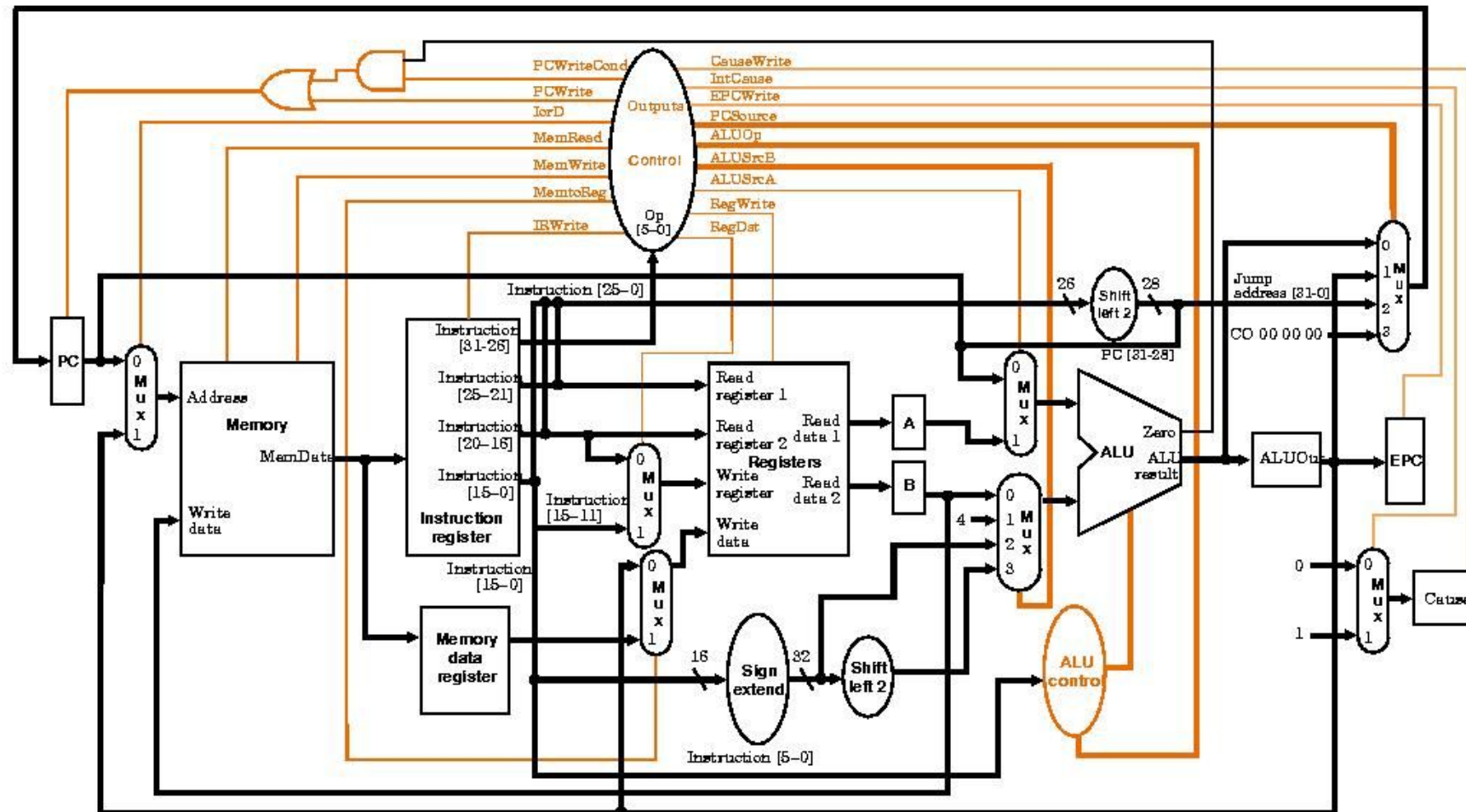
6 – O MODELO DE ACESSO À MEMÓRIA

MAPA MEMÓRIA MIPS

MEMORY ALLOCATION



ORGANIZAÇÃO MIPS MONOCICLO



ORGANIZAÇÃO MIPS MULTICICLO

